TEHNOLOGII WEB

Lucrarea de laborator nr. 3

Componente React, props și state

Proiect: City Explorer

1. Despre lucrare

Lucrarea introduce biblioteca **React** și modul în care o interfață este construită din bucăți independente, care primesc date și reacționează la acțiunile utilizatorului.

Se parcurg componentele și sintaxa JSX, transmiterea datelor prin props, starea locală a unei componente, randarea listelor și tratarea evenimentelor. Lucrul se desfășoară în proiectul creat cu Vite, iar rezultatul se vede de acum în pagină, nu în consolă.

1.1 Cum se citesc blocurile de cod

Deasupra fiecărui bloc de cod este scris ce se face cu el. Nu tot codul tipărit aici ajunge în proiect.

Marcaj	Ce înseamnă
ÎN TERMINAL	Comandă, nu cod. Se scrie în terminal.
FIȘIER NOU	Se creează fișierul de la calea alăturată și se copiază tot conținutul blocului.
SE MODIFICĂ	Fișier existent. Se schimbă doar partea arătată, dacă nu scrie altfel în titlu.
NU SE MODIFICĂ	Cod generat automat. Apare doar ca să se vadă legătura dintre fișiere.
DOAR EXEMPLU	Ilustrează o noțiune. Nu se copiază; forma completă se scrie la exerciții.
REZULTAT	Ce trebuie să apară pe ecran sau în consolă; marcat și „Rezultat așteptat în consolă”. Nu se scrie nicăieri.
STRUCTURĂ	Lista fișierelor lucrării, pentru orientare.

Căile sunt date de la rădăcina proiectului creat cu Vite.

Interfața obținută la finalul lucrării este prezentată în figura 1.

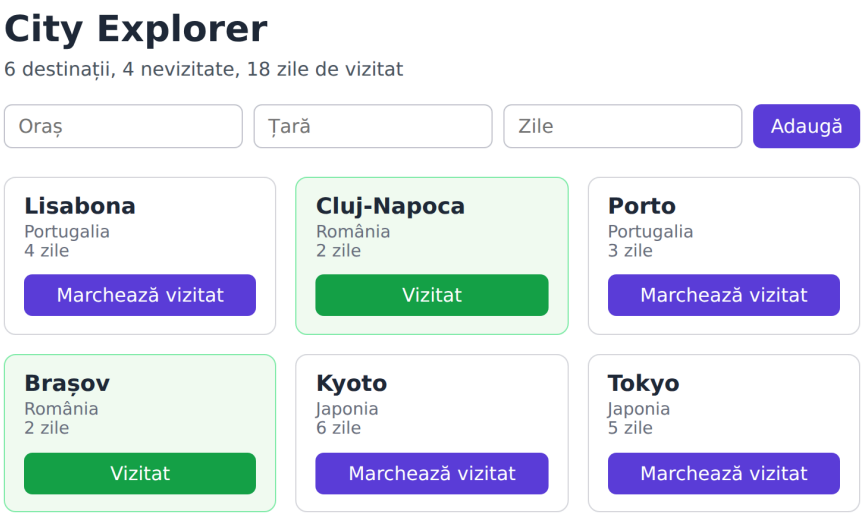


Figura 1. *Aplicația la finalul lucrării.*

Punct de plecare: Lucrarea nu depinde de cea precedentă. Secțiunea 2 conține tot ce trebuie pentru a porni de la zero: crearea proiectului și fișierul cu date. Cine are deja proiectul din lucrarea 2 îl folosește mai departe.

2. Punctul de plecare

Dacă proiectul din lucrarea precedentă există, se deschide și se trece la secțiunea 3. Altfel, se creează acum, în directorul `city-explorer`:

ÎN TERMINAL

```
npm create vite@latest
```

Se răspunde: numele `app`, framework-ul **React**, varianta **JavaScript** (implicit este selectat TypeScript, iar JavaScript este a treia opțiune, deci se coboară de două ori cu săgeata ↓), apoi **No** la instalarea automată. Urmează:

ÎN TERMINAL

```
cd app
npm install
npm run dev
```

Se creează directoarele `src/data`, `src/utils` și `src/components`, iar în `src/data` fișierul cu datele aplicației, identic pentru toată lumea:

FIȘIER NOU `src/data/destinatii.js`

```
export const destinatii = [
  { id: '1', oras: 'Lisabona', tara: 'Portugalia', zile: 4, vizitat: false },
  { id: '2', oras: 'Cluj-Napoca', tara: 'România', zile: 2, vizitat: true },
  { id: '3', oras: 'Porto', tara: 'Portugalia', zile: 3, vizitat: false },
  { id: '4', oras: 'Brașov', tara: 'România', zile: 2, vizitat: true },
  { id: '5', oras: 'Kyoto', tara: 'Japonia', zile: 6, vizitat: false },
  { id: '6', oras: 'Tokyo', tara: 'Japonia', zile: 5, vizitat: false },
]
```

Se creează și `src/utils/destinatii.js`, cu funcțiile de mai jos. Sunt cele scrise în lucrarea precedentă; cine le are deja nu trebuie să le rescrie:

FIȘIER NOU **`src/utils/destinatii.js`**

```
export const getNevizitate = (list) => list.filter((d) => !d.vizitat)

export const totalZile = (list) => list.reduce((total, d) => total + d.zile, 0)

export const marcheazaVizitat = (list, id) =>
  list.map((d) => (d.id === id ? { ...d, vizitat: !d.vizitat } : d))

export const adaugaDestinatie = (list, destinatie) => [...list, destinatie]
```

Fișierul `src/App.css`, generat de Vite, rămâne deocamdată cum este. Se ocupă de el exercițiul 1, care îl înlocuiește.

Verificare. La adresa `http://localhost:5173` se deschide pagina generată de Vite, iar în Explorer există cele trei directoare noi, cu cele două fișiere de date.

3. Componentele: cum este construită pagina

O **componentă** este o funcție obișnuită, scrisă într-un fișier separat, care întoarce bucata de pagină pe care o desenează. Pagina finală este suma acestor bucăți (figura 2).

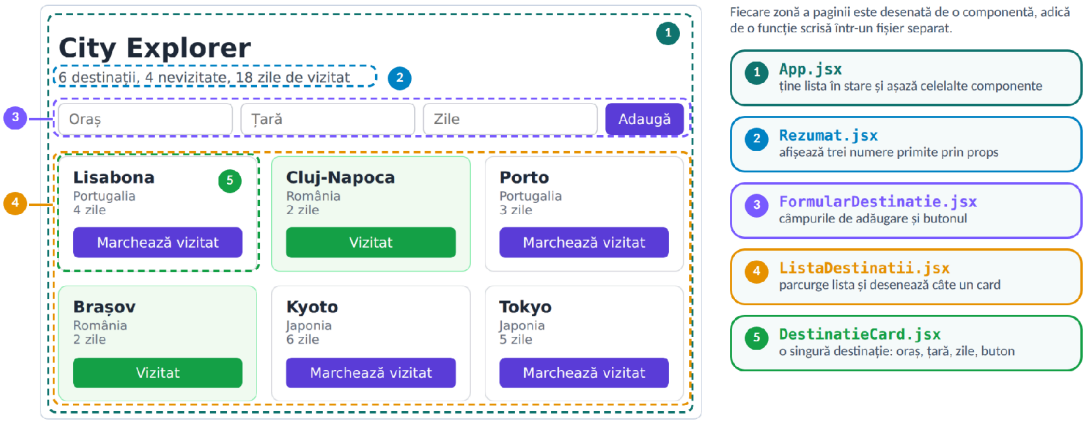


Figura 2. *Ce parte din pagină desenează fiecare componentă.*

Componentele se folosesc una pe alta, ca niște etichete HTML noi, iar aplicația capătă o structură de arbore. Punctul de pornire este `index.html`, care conține un singur element gol, iar `main.jsx` îi spune lui React să deseneze componenta `App` înăuntrul lui (figura 3).

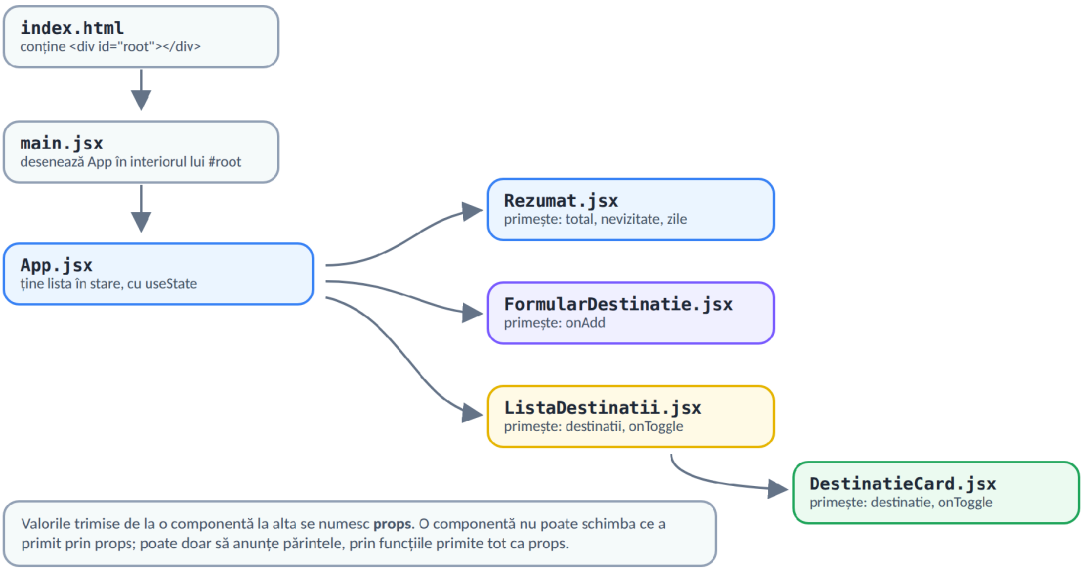


Figura 3. *Ordinea în care se cheamă fișierele și ce primește fiecare componentă.*

Codul care leagă pagina de aplicație a fost generat la crearea proiectului și nu se modifică:

NU SE MODIFICĂ `index.html`

```
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
```

NU SE MODIFICĂ `src/main.jsx`

```
import { createRoot } from 'react-dom/client'
import App from './App.jsx'
import './index.css'

createRoot(document.getElementById('root')).render(<App />)
```

De aici încolo, fiecare fișier are o sarcină proprie. Până acum s-a lucrat într-un singur fișier, `main.jsx`; începând cu această lucrare, interfața se scrie în altă parte:

Fișier	Ce conține	Ce nu conține
`src/main.jsx`	Doar legătura dintre pagină și aplicație: importul lui `App` și linia cu `createRoot`. Sub ea rămân, dacă există, apelurile `console.log` din lucrarea precedentă.	Nicio componentă. Aici nu se scrie `function App`, `function Rezumat` și niciun JSX în afara lui `<App />`.
`src/App.jsx`	Componenta `App`: titlul paginii și etichetele celorlalte componente. Mai târziu, starea aplicației.	Componentele propriu-zise, care stau fiecare în fișierul ei.
`src/components/*.jsx`	Câte o componentă per fișier, exportată cu `export default`.	Apeluri `createRoot` sau cod care rulează la încărcare.

Atenție: Cea mai frecventă greșeală de la începutul lucrării este scrierea componentei `App` tot în `main.jsx`, acolo unde s-a lucrat la laboratorul precedent. Pentru că `main.jsx` o importă deja din `App.jsx`, numele ar fi declarat de două ori în același fișier, iar Vite oprește aplicația cu mesajul `Uncaught SyntaxError: Identifier 'App' has already been declared`. Componenta se mută în `src/App.jsx`, iar în `main.jsx` rămâne doar importul.

Cea mai simplă componentă posibilă arată astfel. Se salvează în `src/components/Rezumat.jsx`:

FIȘIER NOU `src/components/Rezumat.jsx`

```
export default function Rezumat() {
  return <p className="rezumat">Aici va apărea rezumatul.</p>
}
```

Ca să apară pe ecran, componenta trebuie importată și scrisă ca etichetă în altă componentă. O componentă există pe pagină doar dacă cineva o folosește:

SE MODIFICĂ `src/App.jsx, se înlocuiește tot conținutul`

```
import Rezumat from './components/Rezumat.jsx'
import './App.css'

export default function App() {
  return (
    <main>
      <h1>City Explorer</h1>
      <Rezumat />
    </main>
  )
}
```

Verificare. În pagină apar titlul și textul din `Rezumat`. Dacă linia `<Rezumat />` se șterge, textul dispare, deși fișierul rămâne pe disc.

Atenție: Numele unei componente începe obligatoriu cu literă mare. `<Rezumat />` este o componentă, `<rezumat />` ar fi interpretat ca etichetă HTML necunoscută și nu ar afișa nimic.

3.1 JSX

Ce arată ca HTML în interiorul funcției se numește **JSX**. Nu este nici HTML, nici text: este transformat, înainte de a ajunge în browser, în apeluri de funcții. De aceea fișierele au extensia `.jsx` și de aceea proiectul are nevoie de plugin-ul React, instalat odată cu el.

Patru reguli acoperă aproape tot ce se scrie în JSX:

- acoladele înseamnă „aici urmează cod JavaScript”: `{oras}` afișează valoarea variabilei, iar înăuntru poate sta orice expresie, inclusiv un operator ternar;
- atributul de clasă se scrie `className`, nu `class`, pentru că `class` este cuvânt rezervat în JavaScript;
- o componentă întoarce **un singur** element. Dacă sunt mai multe, se învelesc într-un `<div>` sau într-o pereche goală, `<>` și `</>`;
- etichetele fără conținut se închid singure: `<input />`, `<br />`.

DOAR EXEMPLU

```
const oras = 'Lisabona'
const vizitat = true

return (
  <>
    <h3>{oras}</h3>
    <p>{vizitat ? 'Vizitat' : 'De vizitat'}</p>
    <p>{2 + 3} zile</p>
  </>
)
```

3.2 Stilurile

Regulile CSS sunt aceleași ca în lucrarea 1: aceiași selectori, același model al cutiei, aceleași Flexbox și Grid. Se schimbă doar locul lor și felul în care ajung în pagină. Nu mai există o etichetă `<link>` în `index.html`; fișierele de stil se aduc în cod printr-un `import`, la fel ca o funcție:

DOAR EXEMPLU

```
// src/main.jsx
import './index.css'   // stilurile generale: fontul, fundalul, marginile
paginii

// src/App.jsx
import './App.css'     // stilurile componentelor
```

Ambele fișiere au fost generate la crearea proiectului, împreună cu importurile lor, deci nu trebuie adăugat niciun import. Conținutul lor însă este cel al paginii de prezentare a lui Vite și se înlocuiește integral, amândouă, la exercițiul 1a. Importurile rămân pe loc.

Legătura dintre o componentă și o regulă CSS se face prin clase, exact ca în HTML. Atributul `className` pune pe element clasa pe care o caută selectorul:

DOAR EXEMPLU

```
// în src/components/DestinatieCard.jsx
<article className="card">
  <h3>Lisabona</h3>
</article>

/* în src/App.css */
.card {
  border: 1px solid #d8dbe0;
  border-radius: 10px;
  padding: 0.9rem 1rem;
}
```

Pentru că `className` primește un șir obișnuit, valoarea lui poate fi calculată. Un operator ternar adaugă o a doua clasă doar atunci când condiția este adevărată:

DOAR EXEMPLU

```
<article className={destinatie.vizitat ? 'card vizitat' : 'card'}>
```

Elementul are atunci două clase, `card` și `vizitat`. În CSS, regula se scrie pentru combinația lor, `.card.vizitat`, fără spațiu între ele, ceea ce înseamnă „elementul care are ambele clase”. Ea se adaugă peste cea de la `.card` și o poate suprascrie. Aceasta este modalitatea obișnuită de a schimba aspectul unui element în funcție de date.

Atenție: Stilurile nu sunt izolate pe componente. O regulă scrisă pentru `.card` în `App.css` se aplică oricărui element cu clasa `card`, din orice componentă a aplicației. De aceea clasele primesc nume legate de rolul lor, `card`, `lista`, `rezumat`, nu de aspect, `albastru`, `mare`.

4. props: datele primite de la părinte

O componentă devine utilă atunci când primește date din exterior. Valorile se trimit ca atributele unei etichete HTML și ajung în componentă printr-un singur parametru, numit **props**. De obicei, proprietățile se extrag direct în parametri, cu destructuring.

SE MODIFICĂ `src/components/Rezumat.jsx`

```
// componenta primește trei valori
export default function Rezumat({ total, nevizitate, zile }) {
  return (
    <p className="rezumat">
      {total} destinații, {nevizitate} nevizitate, {zile} zile de vizitat
    </p>
  )
}
```

SE MODIFICĂ `src/App.jsx`

```
// părintele le trimite; valorile numerice se pun în acolade
<Rezumat total={6} nevizitate={4} zile={18} />
```

Verificare. În pagină apare „6 destinații, 4 nevizitate, 18 zile de vizitat”. Dacă una dintre valori se schimbă în `App.jsx`, textul se schimbă la salvare.

Atenție: O componentă **nu poate modifica** ce a primit prin props. O atribuire de forma `total = 10` în interiorul componentei nu are niciun efect asupra părintelui. Dacă trebuie schimbat ceva, se cere părintelui, printr-o funcție primită tot prin props, așa cum se vede în secțiunea 6.

5. state: datele care se schimbă

Props vin de sus și nu se pot schimba. Ce se schimbă în timp, prin acțiunile utilizatorului, se ține în **state**, cu ajutorul funcției `useState`. Diferența dintre cele două este prezentată în figura 4.

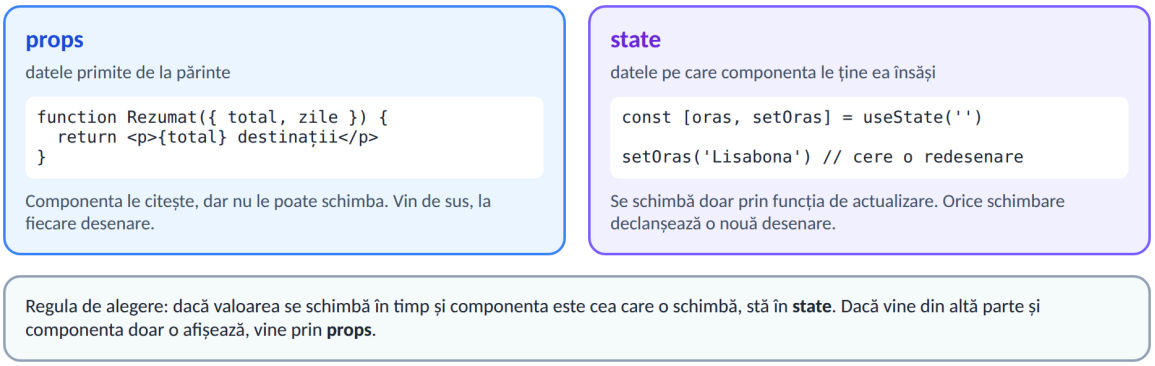


Figura 4. *props față de state.*

`useState` se importă din React și întoarce întotdeauna două lucruri: valoarea curentă și funcția cu care se cere schimbarea ei. Se scriu împreună, între paranteze drepte:

DOAR EXEMPLU

```
import { useState } from 'react'

export default function Contor() {
  const [numar, setNumar] = useState(0) // 0 este valoarea de pornire

  return (
    <button type="button" onClick={() => setNumar(numar + 1)}>
      Ai dat click de {numar} ori
    </button>
  )
}
```

La fiecare apăsare, `setNumar` cere o valoare nouă, iar React execută din nou funcția componentei și redesenează ce s-a schimbat. Ciclul complet este în figura 5.

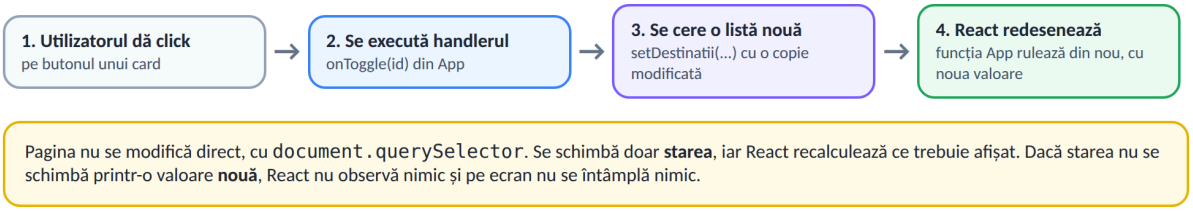


Figura 5. *Ce se întâmplă de la click până la afișare.*

Atenție: Valoarea din stare **nu** se modifică direct: `numar = numar + 1` nu face nimic. Singurul mod de a o schimba este funcția de actualizare.

Când noua valoare depinde de cea veche, forma sigură este `setNumar((curent) => curent + 1)`. React trimite valoarea curentă ca parametru, iar codul funcționează corect chiar dacă se cer mai multe actualizări una după alta.

Pentru liste și obiecte, regula are o consecință importantă: nu se modifică lista existentă, ci se construiește una nouă. Este exact tiparul din lucrarea precedentă, iar funcțiile din `src/utils/destinatii.js` îl respectă deja:

DOAR EXEMPLU

```
const [destinatii, setDestinatii] = useState(destinatiiInitiale)

// corect: marcheazaVizitat întoarce o listă nouă
setDestinatii((curent) => marcheazaVizitat(curent, id))

// greșit: React nu observă nicio schimbare, pagina rămâne la fel
destinatii[0].vizitat = true
```

6. Liste, key și evenimente

O listă de date se transformă în listă de componente cu aceeași metodă `map` din lucrarea precedentă, doar că rezultatul nu mai este text, ci JSX:

DOAR EXEMPLU **`tiparul pentru src/components/ListaDestinatii.jsx`**

```
import DestinatieCard from './DestinatieCard.jsx'

export default function ListaDestinatii({ destinatii }) {
  return (
    <div className="lista">
      {destinatii.map((destinatie) => (
        <DestinatieCard key={destinatie.id} destinatie={destinatie} />
      ))}
    </div>
  )
}
```

Atributul `key` nu este decorativ. React îl folosește ca să recunoască elementele între două desenări, atunci când lista se schimbă. Se pune **identificatorul** elementului, nu poziția lui în listă (figura 6).

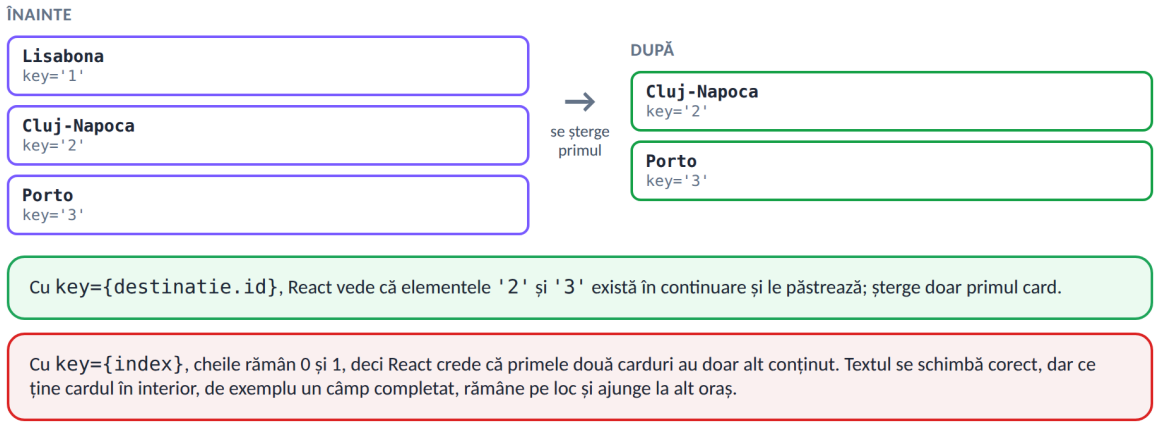


Figura 6. *De ce key trebuie să fie identificatorul, nu indicele.*

6.1 Evenimente

Reacția la acțiunile utilizatorului se scrie ca atribut, în JSX. Atributul primește o **funcție**, nu rezultatul apelării ei:

DOAR EXEMPLU

```
// corect: funcția este apelată la click
<button onClick={() => onToggle(id)}>Marchează vizitat</button>

// greșit: funcția se execută imediat, la desenare
<button onClick={onToggle(id)}>Marchează vizitat</button>
```

Cele mai folosite atribute de acest tip sunt `onClick`, pentru butoane, `onChange`, pentru câmpurile de formular, și `onSubmit`, pentru formularul întreg. Denumirile se scriu cu literă mare în interior, spre deosebire de HTML.

6.2 Evenimentele urcă, datele coboară

Un card nu poate schimba singur lista, pentru că lista nu îi aparține: ea stă în componenta părinte. Cardul primește prin props o **funcție** și o apelează atunci când se dă click. Părintele este cel care schimbă starea.

DOAR EXEMPLU **`tiparul pentru src/App.jsx`**

```
// în App.jsx: funcția care chiar schimbă starea
function handleToggle(id) {
  setDestinatii((curent) => marcheazaVizitat(curent, id))
}

<ListaDestinatii destinatii={destinatii} onToggle={handleToggle} />
```

DOAR EXEMPLU **`tiparul pentru src/components/DestinatieCard.jsx`**

```
// în DestinatieCard.jsx: cardul doar anunță ce s-a întâmplat
export default function DestinatieCard({ destinatie, onToggle }) {
  return (
    <article className={destinatie.vizitat ? 'card vizitat' : 'card'}>
      <h3>{destinatie.oras}</h3>
      <button type="button" onClick={() => onToggle(destinatie.id)}>
        {destinatie.vizitat ? 'Vizitat' : 'Marchează vizitat'}
      </button>
    </article>
  )
}
```

Funcția trece prin `ListaDestinatii`, care o primește și o dă mai departe fiecărui card. Datele coboară prin props, iar evenimentele urcă prin funcțiile primite tot ca props.

Cu aceasta, drumul complet a fost parcurs o dată: pagina împărțită în componente, date transmise prin props, o valoare ținută în stare și schimbată la click, o listă desenată cu `map` și `key`. Exemplele au folosit un contor și un rezumat; exercițiile aplică aceleași noțiuni pe destinațiile proiectului.

7. Exerciții

Exercițiile construiesc interfața aplicației **City Explorer**. Se lucrează în proiectul din secțiunea 2, iar fișierele componentelor se creează în `src/components`. Verificarea se face vizual, în browser, la `http://localhost:5173`.

Fișierele lucrării:

Fișier	Ce conține
`src/components/DestinatieCard.jsx`	o singură destinație, ca un card
`src/components/ListaDestinatii.jsx`	lista de carduri
`src/components/Rezumat.jsx`	cele trei numere de sus
`src/components/FormularDestinatie.jsx`	câmpurile de adăugare
`src/App.jsx`	starea aplicației și legarea componentelor
`src/index.css`	stilurile paginii, la punctul 1a
`src/App.css`	stilurile componentelor: cadrul la punctul 1a, lista și cardul la 1e

Exercițiul 1. Cardurile, lista și stilurile

a) Punctul de plecare pentru stiluri. Proiectul vine cu două fișiere de stil, `src/index.css` și `src/App.css`, amândouă pline cu regulile paginii de prezentare generate de Vite. Conținutul lor se șterge și se înlocuiește cu cel de mai jos.

Întâi `src/index.css`, care se ocupă de pagină în ansamblu, fontul și fundalul:

SE MODIFICĂ `src/index.css, se înlocuiește tot conținutul`

```
body {
  font-family: system-ui, sans-serif;
  margin: 0;
  background: #fff;
  color: #1f2937;
}
```

Atenție: Pasul acesta nu se sare. Regulile lăsate de Vite în `index.css` centrează tot textul paginii, îngustează conținutul la o coloană și desenează două linii verticale pe margini. Aspectul rezultat nu seamănă cu nicio captură din lucrare, iar cauza nu se vede în `App.css`, unde va fi căutată.

Apoi `src/App.css`, cu regulile componentelor. Cele de mai jos se ocupă de cadrul paginii, de formular și de butonul obișnuit, și sunt date pentru că nu sunt subiectul lucrării. Se copiază ca atare, nu se modifică:

SE MODIFICĂ `src/App.css, se înlocuiește tot conținutul`

```
main {
  max-width: 720px;
  margin: 2rem auto;
  padding: 0 1rem;
}
```

```
h1 {
  font-size: 1.9rem;
  margin-bottom: 0.4rem;
}

.rezumat {
  color: #4b5563;
  margin-top: 0;
  margin-bottom: 1.2rem;
}

.formular {
  display: flex;
  gap: 0.6rem;
  margin-bottom: 1.5rem;
}

.formular input {
  flex: 1;
  padding: 0.5rem 0.7rem;
  border: 1px solid #c3c7cf;
  border-radius: 7px;
  font: inherit;
  min-width: 0;
}

button {
  padding: 0.5rem 0.9rem;
  border: none;
  border-radius: 7px;
  background: #5b3fd9;
  color: #fff;
  font: inherit;
  cursor: pointer;
}

.info {
  padding: 0.8rem;
  border-radius: 9px;
  background: #f3f4f6;
}
```

După acest punct pagina este goală și albă, cu fundal alb și text negru obișnuit: conținutul generat de Vite a dispărut, iar din noile reguli niciuna nu are încă pe ce să se aplice, pentru că nu există elemente cu clasele `rezumat`, `formular` sau `info`. Stilurile listei și ale cardului lipsesc din fișier intenționat; se scriu la punctul **e**.

b) Cardul. Să se creeze componenta `DestinatieCard`, care primește prin props un obiect `destinatie` și afișează orașul într-un `<h3>`, țara și numărul de zile în două paragrafe, cu clasele `tara` și `zile`. Elementul exterior este un `<article>` cu clasa `card`, iar dacă destinația este vizitată primește și clasa `vizitat`, printr-un operator ternar.

FIȘIER NOU `src/components/DestinatieCard.jsx`

```
export default function DestinatieCard({ destinatie }) {
  const { oras, tara, zile, vizitat } = destinatie

  return (
    <article className={vizitat ? 'card vizitat' : 'card'}>
      {/* titlul, țara și zilele */}
    </article>
  )
}
```

c) Lista. Să se creeze componenta `ListaDestinatii`, care primește prin props lista `destinatii` și desenează câte un `DestinatieCard` pentru fiecare element, cu `map` și cu `key` pus pe identificator. Cardurile se învelesc într-un `<div>` cu clasa `lista`. Dacă lista este goală, se afișează în loc un paragraf cu clasa `info` și textul „Nu există nicio destinație.”

d) Legarea. În `App.jsx` se importă lista și componenta, apoi componenta se scrie ca etichetă. Până acum conținutul fișierului era cel de la secțiunea 3; acum devine:

SE MODIFICĂ `src/App.jsx`

```
import { destinatii } from './data/destinatii.js'
import ListaDestinatii from './components/ListaDestinatii.jsx'
import './App.css'

export default function App() {
  return (
    <main>
      <h1>City Explorer</h1>
      <ListaDestinatii destinatii={destinatii} />
    </main>
  )
}
```

După acest punct, pe pagină apar cele șase destinații, una sub alta, fără chenar și fără culoare: elementele există, dar nu există încă nicio regulă pentru clasele lor.

e) Stilurile listei și ale cardului. Să se adauge la finalul fișierului `src/App.css` cele șapte reguli de mai jos, pentru clasele puse la punctele **b** și **c**. Tabelul dă selectorul și proprietățile care trebuie scrise; valorile se caută și se aleg astfel încât rezultatul să semene cu figura 7:

Selector	Ce trebuie obținut	Proprietăți

`.lista`	Cardurile așezate în grilă, pe coloane care se adaptează la lățimea ferestrei: fiecare coloană de cel puțin 200px, câte încap pe un rând, cu spațiu egal între ele.	`display: val;`
		`grid-template-columns: val;`
		`gap: val;`

`.card`	Chenar subțire gri, colțuri ușor rotunjite, spațiu între text și marginea cardului.	`border: val;`
		`border-radius: val;`
		`padding: val;`

`.card h3`	Titlul apropiat de țara de dedesubt: marginile implicite ale lui `<h3>` se anulează, rămâne doar un spațiu mic sub el.	`margin: val;`

Selector	Ce trebuie obținut	Proprietăți

`.card .tara, .card .zile`	Cele două paragrafe fără margini, gri și puțin mai mici decât textul obișnuit. Un singur bloc, cu doi selectori despărțiți prin virgulă.	`margin: val;`
		`color: val;`
		`font-size: val;`

`.card button`	Butonul pe toată lățimea cardului, desprins de text printr-un spațiu deasupra.	`margin-top: val;` `width: val;`
`.card.vizitat`	Fundal verde foarte deschis și chenar verde, doar pentru cardul care are ambele clase.	`background: val;` `border-color: val;`
`.card.vizitat button`	Butonul verde, în cardul deja vizitat.	`background: val;`

Indicație: Valorile exacte nu sunt impuse: două nuanțe apropiate de verde sunt amândouă corecte. Se verifică efectul, nu codul culorii. Proprietățile necunoscute se caută în documentația MDN: lista completă, cu valorile posibile și exemple, în [8], iar pentru grilă, explicația lui `grid-template-columns` cu `repeat`, `auto-fit` și `minmax`, în [9].

Atenție: Ultimele două selectoare se scriu fără spațiu între clase, `.card.vizitat`. Scris cu spațiu, `.card .vizitat`, selectorul caută un element cu clasa `vizitat` **în interiorul** cardului, care nu există, și regula nu se aplică.

Verificare. Pe pagină apar cele șase carduri, așezate în grilă, două dintre ele, Cluj-Napoca și Brașov, cu fundal verde (figura 7). Dacă un card arată altfel, cauza este fie numele clasei din componentă, fie selectorul din CSS. În consolă nu există avertismentul despre `key`.



Figura 7. *Rezultatul exercițiului 1: cardurile cu stilurile scrise la punctul e.*

Exercițiul 2. Rezumatul și starea

a) Rezumatul. Să se creeze componenta `Rezumat`, care primește prin props trei numere, `total`, `nevizitate` și `zile`, și le afișează într-un paragraf cu clasa `rezumat`, sub titlu: „6 destinații, 4 nevizitate, 18 zile de vizitat”.

b) Starea. În `App.jsx`, lista nu se mai folosește direct din fișierul de date, ci se pune în stare, ca să poată fi modificată:

SE MODIFICĂ `src/App.jsx`

```
import { useState } from 'react'
import { destinatii as destinatiiInitiale } from './data/destinatii.js'

export default function App() {
  const [destinatii, setDestinatii] = useState(destinatiiInitiale)
  // ...
}
```

Cele trei numere trimise lui `Rezumat` se calculează din stare, cu funcțiile din `src/utils/destinatii.js`: `destinatii.length`, `getNevizitate(destinatii).length` și `totalZile(getNevizitate(destinatii))`.

c) Marcarea. Fiecare card primește un buton. La click, destinația își schimbă starea de vizitat:

1. în `App.jsx` se scrie funcția `handleToggle(id)`, care apelează `setDestinatii` cu rezultatul lui `marcheazaVizitat`, folosind forma cu funcție: `setDestinatii((curent) => ...)`;
2. funcția se transmite lui `ListaDestinatii` prin props, cu numele `onToggle`, iar de acolo mai departe fiecărui card;
3. în `DestinatieCard`, butonul o apelează la click, cu identificatorul destinației, iar textul lui este „Vizitat” sau „Marchează vizitat”, după caz.

Verificare. Un click pe butonul unui card schimbă atât aspectul cardului, cât și cele trei numere din rezumat, fără reîncărcarea paginii. Rezumatul se actualizează singur, pentru că este calculat din stare la fiecare desenare.

Indicație: Dacă la click nu se întâmplă nimic, cauza este aproape sigur una dintre două: lista a fost modificată direct, în loc să fie construită una nouă, sau `onClick` a primit rezultatul funcției, `onClick={onToggle(id)}`, în loc de funcția însăși.

Exercițiul 3. Formularul de adăugare

Să se creeze componenta `FormularDestinatie`, cu trei câmpuri, pentru oraș, țară și număr de zile, și un buton de trimitere. Elementul `<form>` primește clasa `formular`. Este primul formular din proiect, deci merită parcurs pe pași:

1. fiecare câmp își ține valoarea în stare, cu `useState`, iar în JSX primește `value={oras}` și `onChange={(e) => setOras(e.target.value)}`. Valoarea afișată vine din stare, nu din pagină;
2. formularul întreg primește `onSubmit={handleSubmit}`, iar funcția începe obligatoriu cu `event.preventDefault()`, altfel browserul reîncarcă pagina și starea se pierde;
3. dacă orașul sau țara sunt goale, funcția se oprește, cu `return`, fără să trimită nimic;
4. altfel se apelează funcția primită prin props, `onAdd`, cu un obiect nou care are `id`, `oras`, `tara`, `zile` convertit cu `Number(...)` și `vizitat: false`. Identificatorul se generează cu `crypto.randomUUID()`;
5. după trimitere, cele trei câmpuri se golesc, punând înapoi șirul vid în stare.

În `App.jsx` se scrie funcția `handleAdd`, care adaugă destinația în stare cu `adaugaDestinatie`, și se transmite formularului prin props.

Verificare. După completarea câmpurilor și apăsarea butonului, cardul nou apare la finalul listei, rezumatul crește cu o destinație și cu zilele introduse, iar câmpurile rămân goale, gata pentru următoarea adăugare (figura 8). Un formular trimis cu orașul gol nu adaugă nimic.

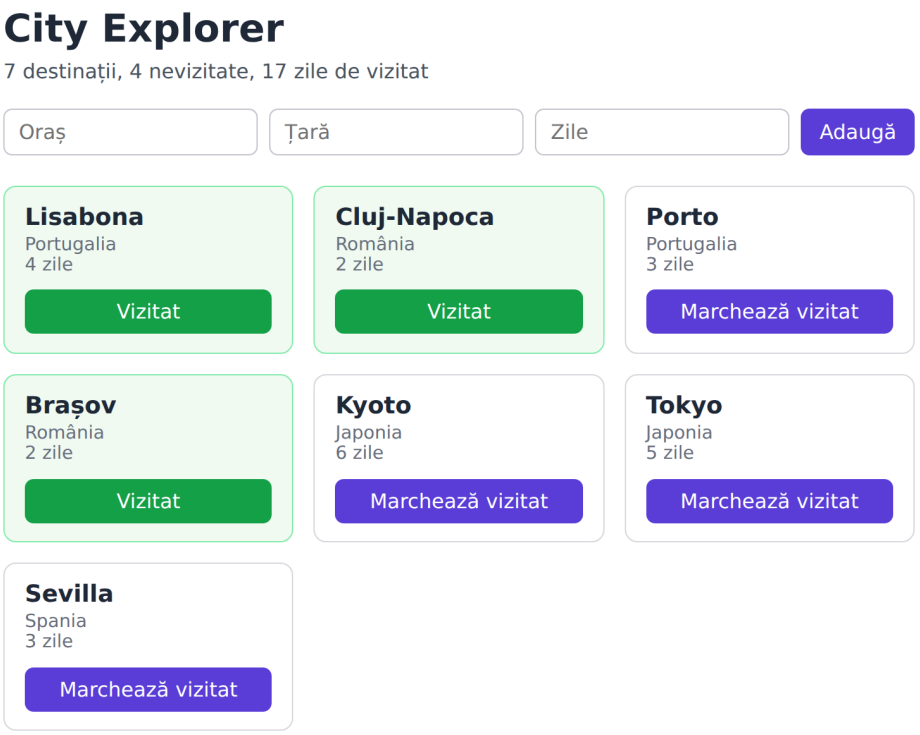


Figura 8. *Aplicația după marcarea Lisabonei și adăugarea Sevillei.*

Indicație: Cerință suplimentară: un buton de ștergere pe fiecare card, care apelează o funcție `onDelete` primită prin props, iar în `App.jsx` scoate destinația din stare cu `stergeDestinatie`.

8. Probleme frecvente

Simptom	Cauză și soluție
Pagina rămâne albă	O eroare de sintaxă în JSX. Se citește mesajul afișat de Vite peste pagină; el indică fișierul și linia.
Tot textul apare centrat, iar pe margini coboară două linii verticale	Conținutul generat de Vite în `src/index.css` nu a fost înlocuit. Se șterge tot și se pun cele cinci linii din exercițiul 1a.
Cardurile apar unul sub altul, nu în grilă	Lipsește regula `.lista` sau `<div>`-ul din `ListaDestinatii` nu are clasa `lista`.
Butonul nu ocupă lățimea cardului	Lipsește regula `.card button`. Scrisă ca o clasă nouă pe buton, nu prinde: se scrie ca selector compus.
„Identifier `App` has already been declared”	Componenta `App` a fost scrisă în `main.jsx`, care o și importă din `App.jsx`. Se mută în `src/App.jsx`; în `main.jsx` rămâne doar importul.
Rezumatul afișează „undefined destinații”	Componenta a fost scrisă ca `<Rezumat />`, fără props. Se transmit cele trei valori: `<Rezumat total={...} nevizitate={...} zile={...} />`.

Simptom	Cauză și soluție
„Adjacent JSX elements must be wrapped”	Componenta întoarce mai multe elemente. Se învelesc în `<>` și `</>`.
Componenta nu apare pe pagină	Nu a fost scrisă ca etichetă în părinte, sau numele începe cu literă mică.
„Each child in a list should have a unique key prop”	Lipsește `key` la `map`. Se adaugă `key={destinatie.id}`.
Click-ul nu face nimic	`onClick` a primit rezultatul funcției. Se scrie `onClick={() => onToggle(id)}`.
Funcția se execută singură, la încărcare	Aceeași cauză: lipsesc parantezele săgeată din jurul apelului.
Starea se schimbă, dar pagina nu	Lista a fost modificată direct. Se construiește o listă nouă, cu `map`, `filter` sau spread.
Pagina se reîncarcă la trimiterea formularului	Lipsește `event.preventDefault()` în `handleSubmit`.
Avertisment: „changing an uncontrolled input to be controlled”	Câmpul a pornit cu `undefined`. Se inițializează starea cu șirul vid, `useState('')`.
Numărul de zile se adună ca text	Câmpurile întorc text. Se convertește cu `Number(...)` înainte de a fi pus în obiect.
„Objects are not valid as a React child”	Se afișează un obiect întreg. Se afișează o proprietate, de exemplu `destinatie.oras`.

9. Bibliografie

[1] Meta Platforms, Inc., *Your First Component*, React Documentation, 2026. [Online]. Disponibil: https://react.dev/learn/your-first-component

[2] Meta Platforms, Inc., *Passing Props to a Component*, React Documentation, 2026. [Online]. Disponibil: https://react.dev/learn/passing-props-to-a-component

[3] Meta Platforms, Inc., *Rendering Lists*, React Documentation, 2026. [Online]. Disponibil: https://react.dev/learn/rendering-lists

[4] Meta Platforms, Inc., *State: A Component's Memory*, React Documentation, 2026. [Online]. Disponibil: https://react.dev/learn/state-a-components-memory

[5] Meta Platforms, Inc., *Updating Arrays in State*, React Documentation, 2026. [Online]. Disponibil: https://react.dev/learn/updating-arrays-in-state

[6] Meta Platforms, Inc., *Responding to Events*, React Documentation, 2026. [Online]. Disponibil: https://react.dev/learn/responding-to-events

[7] Mozilla Foundation, *Arrow function expressions*, MDN Web Docs, 2026. [Online]. Disponibil: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions

[8] Mozilla Foundation, *CSS reference*, MDN Web Docs, 2026. [Online]. Disponibil: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference

[9] Mozilla Foundation, *Basic concepts of grid layout*, MDN Web Docs, 2026. [Online]. Disponibil: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS*grid*layout/Basic*concepts*of*grid*layout